

Building a Web Server with Node.js

A hands-on introduction to HTTP servers, routing, JSON APIs, and unit testing — all without a single external framework to start. Designed for developers new to Node.js who want to understand what happens under the hood before reaching for Express.

PART 1: WEB SERVERS

PART 2: UNIT TESTING



Part 1 — Building a Web Server

In this section, we'll trace the full journey of an HTTP request from browser to server and back, then build a working Node.js API — entirely using built-in modules. No Express, no Spring, no Nginx required.

HTTP Fundamentals

Understand requests, responses, methods, and status codes.

The http Module

Node's built-in server creation without any frameworks.

Routing & JSON

Manually route requests and return structured API data.

What Happens When You Open a URL?

When you navigate to `http://localhost:3000/students`, your browser initiates a full HTTP conversation with the server. Understanding this flow is foundational to everything that follows.



Every web interaction follows this pattern. The server's only job is to receive a request and produce a meaningful response — a concept that scales from simple text replies to full REST APIs.

Anatomy of a Request and Response

HTTP messages have a well-defined structure. Knowing what each part contains helps you write handlers that correctly interpret incoming data and craft accurate outgoing replies.

Request Contains

- **HTTP Method** — GET, POST, PUT, DELETE
- **URL / Path** — e.g., `/students`
- **Headers** — metadata like Content-Type, Authorization
- **Body** — data sent with POST/PUT requests

Response Contains

- **Status Code** — 200, 404, 500, etc.
- **Headers** — Content-Type, Cache-Control, etc.
- **Response Body** — HTML, JSON, plain text, or binary

❏ Key idea: A web server receives requests and produces responses. Every framework, no matter how complex, is built on this simple contract.

The Node.js http Module

Node.js ships with a built-in `http` module — no installation required. You can create a fully functional HTTP server in just a few lines, without Express, Spring, Tomcat, Nginx, or Apache.

Require the module

```
const http = require('http');
```

Create a server

```
const server = http.createServer(  
  (req, res) => {  
    // request handling logic  
  }  
);
```

Start listening

```
server.listen(3000, () => {  
  console.log('Server started');  
});
```

What this means

`http.createServer()` takes a callback — your **request handler** — that fires for every incoming HTTP request. The `listen()` call binds the server to a port (3000 here) and begins accepting connections.

No Express needed

Node itself is the server.

Port 3000

Standard dev convention.

The Request Handler: `req` and `res`

The callback passed to `createServer()` receives two arguments for every request. Together they give you everything you need to read what the client asked and craft your reply.

`req` — Incoming Request

Represents what the client sent. Key properties:

- `req.url` — the path requested
- `req.method` — GET, POST, etc.
- `req.headers` — all request headers

```
console.log(req.method);  
console.log(req.url);
```

`res` — Outgoing Response

Represents what the server sends back. Key methods:

- `res.writeHead()` — set status code + headers
- `res.write()` — write body chunks
- `res.end()` — finish and send the response

```
res.writeHead(200, {  
  'Content-Type': 'text/plain'  
});  
res.end('Hello!');
```

Mental model: `req` → "What did the client ask?" | `res` → "What should the server send back?"

Your First Node.js Web Server

Let's put it all together. This minimal server responds to every request with a plain-text welcome message — a working HTTP server in under 15 lines of JavaScript.

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Welcome to the Student Portal API');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

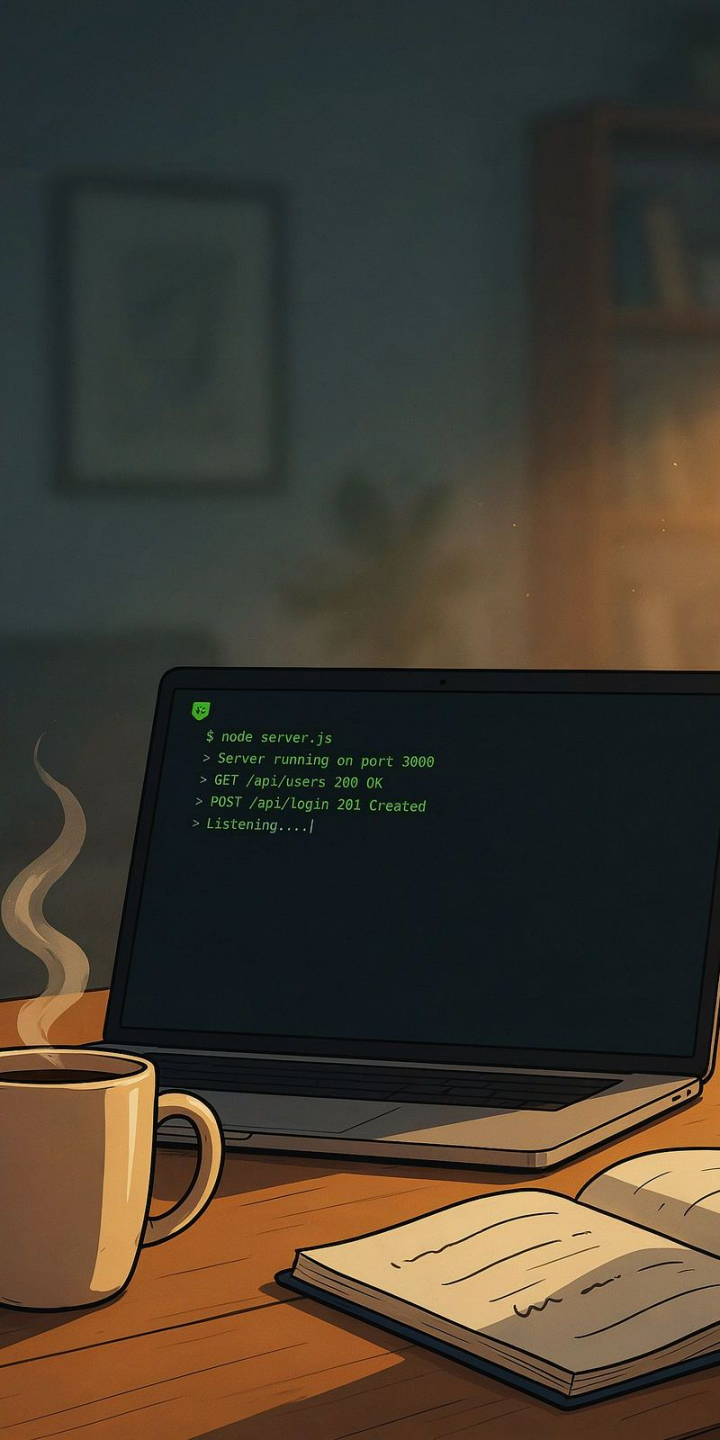
Run the server

```
node server.js
```

Test in browser

```
http://localhost:3000
```

Expected output: Welcome to the
Student Portal API



Understanding HTTP Status Codes

Status codes are how your server communicates the *outcome* of a request. Always set the correct status code — it's part of the HTTP contract and helps clients (and developers) understand what happened.

Code	Meaning	Example Use Case
200	OK	Request succeeded, response body contains the result
201	Created	A new resource was successfully created (POST)
400	Bad Request	Client sent malformed or invalid input
401	Unauthorized	Authentication is required but missing or invalid
404	Not Found	The requested resource does not exist
500	Server Error	An unexpected error occurred in the application

Success response

```
res.writeHead(200, {  
  'Content-Type': 'text/plain'  
});
```

Missing resource

```
res.writeHead(404, {  
  'Content-Type': 'text/plain'  
});
```


Building a Simple Router

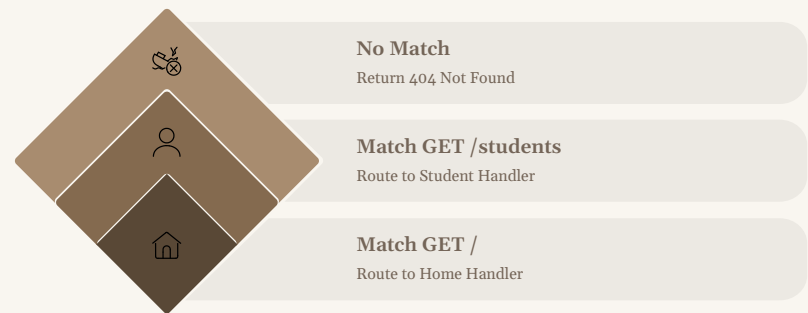
Without a framework like Express, routing is done manually by inspecting `req.method` and `req.url`. It's straightforward for small apps — and understanding it makes Express far less mysterious.

Manual routing logic

```
if (req.url === '/' &&
    req.method === 'GET') {
  // serve home page
}

if (req.url === '/students' &&
    req.method === 'GET') {
  // serve student list
}
```

Conceptual flow



📌 Teaching point: This approach works perfectly for small apps, but chains of if/else if blocks become hard to maintain as your endpoint count grows. That's exactly the problem Express solves.

Returning JSON from Node.js

REST APIs almost always communicate using JSON. Returning JSON requires two things: setting the correct `Content-Type` header and converting your JavaScript objects to a JSON string before sending.

Define your data

```
const students = [  
  { id: 1, name: 'Aditi' },  
  { id: 2, name: 'Rahul' }  
];
```

Send as JSON

```
res.writeHead(200, {  
  'Content-Type': 'application/json'  
});  
res.end(JSON.stringify(students));
```

Client receives

```
[  
  { "id": 1, "name": "Aditi" },  
  { "id": 2, "name": "Rahul" }  
]
```

JSON.stringify() is required

`res.end()` expects a string or Buffer. JavaScript objects must be serialized first — passing a raw object will not work.

Complete Student Portal API

Here is the full implementation combining routing, JSON responses, and proper status codes into a single working server. Test it with curl or your browser.

```
const http = require('http');
const students = [
  { id: 1, name: 'Aditi' },
  { id: 2, name: 'Rahul' }
];

const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Welcome to Student Portal API');
  } else if (req.url === '/students' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(students));
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('Not Found');
  }
});

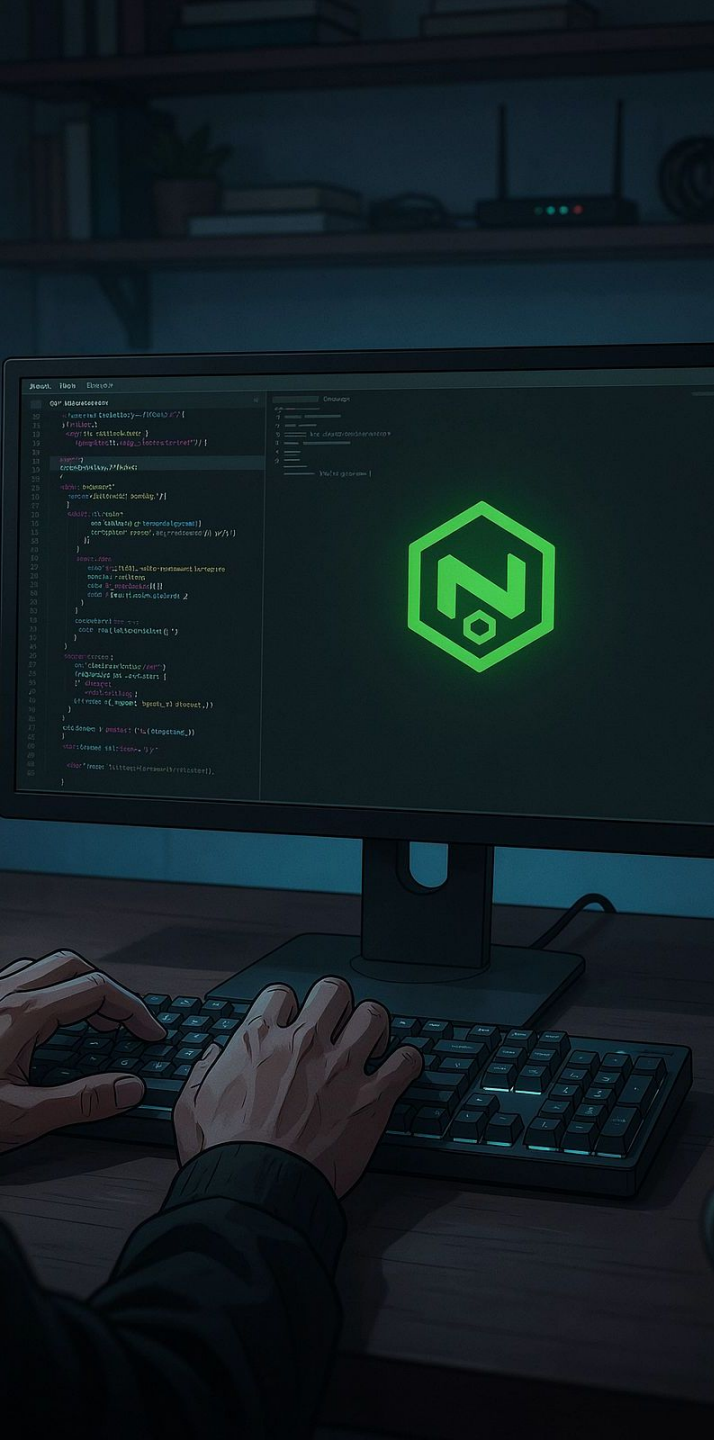
server.listen(3000);
```

Test root

```
curl http://localhost:3000/
```

Test students

```
curl http://localhost:3000/students
```



From Node http to Express

Our working router uses if/else if chains. That's fine for 2–3 routes. Imagine maintaining 50. Express was built precisely to solve this problem — it wraps Node's http module with a clean, expressive API.

Raw Node.js (verbose)

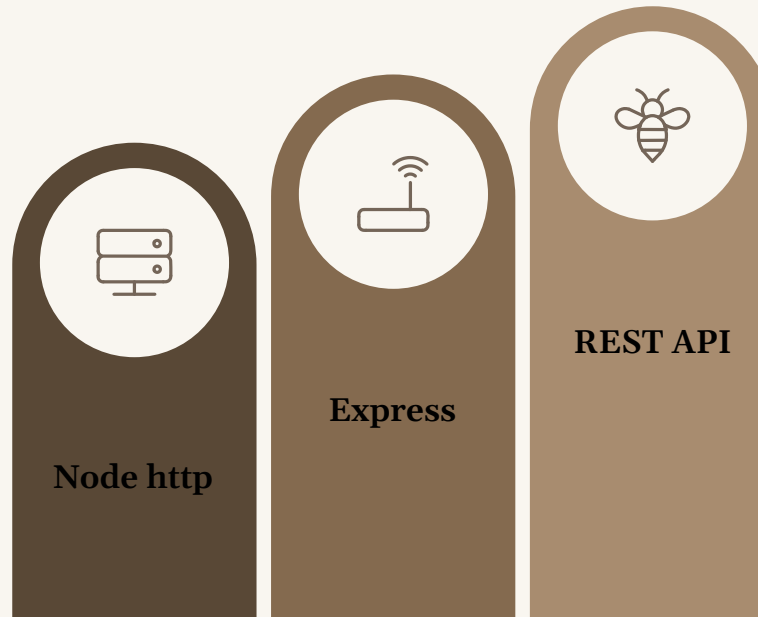
```
if (
  req.url === '/students' &&
  req.method === 'GET'
){
  // handler
}
```

Repeating this pattern for every route leads to deeply nested, fragile code.

Express (clean)

```
app.get('/students',
  (req, res) => {
    // handler
  }
);
```

Express adds routing, middleware, request parsing, error handling, and clear application structure on top of Node's HTTP capabilities.



Node.js HTTP vs. Spring Boot

For developers coming from the Java ecosystem, this comparison makes the mental model click. Both platforms solve the same fundamental problem using the same HTTP principles — they just get there differently.

Node.js Stack

- Node.js runtime
- http module (built-in)
- Request Handler callback
- HTTP Response

Spring Boot Stack

- Spring Boot application
- Embedded Tomcat (built-in)
- Spring MVC dispatcher
- Controller → Response

What They Share

- Listen on a port
- Receive HTTP requests
- Route to application logic
- Return HTTP responses

☐ Takeaway: Express is to Node's HTTP module what Spring MVC is to the underlying servlet/container model. Frameworks add structure — they don't change the fundamental request/response contract.

Part 2 — Node.js Unit Testing with Mocha

Manually clicking through an app to verify it works is slow, error-prone, and impossible to scale. In this section, we introduce automated testing with Mocha so you can validate your server logic quickly and confidently after every code change.



Confidence After Changes

Tests verify that existing functionality still works whenever you modify code.



Speed

Automated tests run in milliseconds — far faster than any manual workflow.

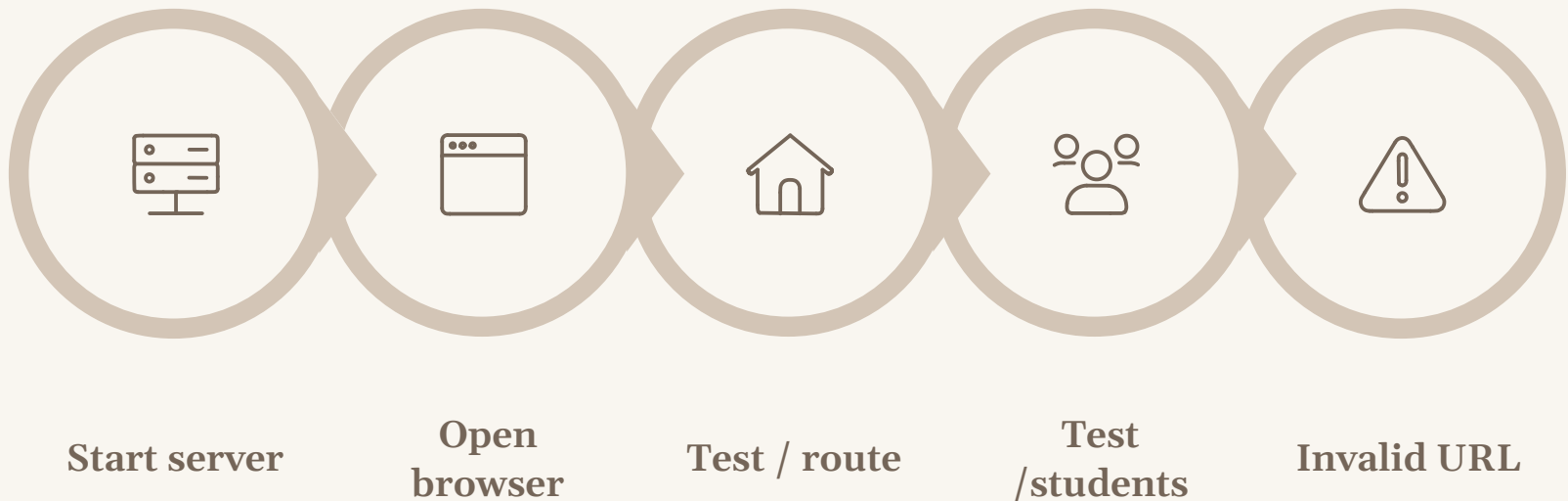


Early Bug Detection

Catch regressions the moment they're introduced, not after deployment.

Why Do We Need Tests?

Consider the manual testing workflow every time you change a single line of code. This cycle is tedious, slow, and — more importantly — human memory is unreliable. Steps get skipped.



The manual problem

Every code change requires repeating the entire manual checklist. As your API grows to 20+ endpoints, this becomes impractical — and humans make mistakes.

Automated solution

Write tests once. Run them instantly with `npm test`. The output is a clear **PASS** or **FAIL** — no guesswork, no missed cases.

What Is Mocha?

Mocha is a JavaScript **test runner** — it finds your test files, executes them, and reports results. It is not an assertion library. Think of it as the scaffolding that organizes and runs your tests.

1 Finds test files

Automatically discovers files inside the `test/` directory.

2 Organizes suites

Groups tests using `describe()` blocks for readable output.

3 Reports results

Clear pass/fail output with helpful error messages for failures.

4 Supports hooks

Provides `before`, `after`, `beforeEach`, and `afterEach` for setup and teardown.

- ❑ Mocha is NOT an assertion library. For this lab we use Node's built-in `assert` module. Later, pairing Mocha with **Chai** gives you more expressive, readable assertions.



Installing and Running Mocha

Getting Mocha up and running takes less than two minutes. It installs as a dev dependency and integrates with the standard `npm test` command.

1. Create your project

```
mkdir node-testing-demo
cd node-testing-demo
npm init -y
```

2. Install Mocha

```
npm install --save-dev mocha
```

3. Update package.json

```
{
  "scripts": {
    "test": "mocha"
  }
}
```

4. Create your test file

```
test/
├── sample.test.js
```

5. Run tests

```
npm test
```

Mocha automatically discovers all test files inside the `test/` directory. No configuration file is required for basic usage — convention over configuration.

Writing Your First Test

Mocha tests are organized using two functions: `describe()` to group related tests, and `it()` to define individual test cases. Node's built-in `assert` module handles the actual value comparisons.

test/sample.test.js

```
const assert = require('assert');

describe('Math operations', () => {

  it('should add two numbers correctly',
    () => {
      assert.strictEqual(2 + 3, 5);
    }
  );

});
```

Key building blocks

`describe()`

Groups related test cases into a named suite. Nesting is allowed for sub-groups.

`it()`

Defines a single test case. The string description should read like a sentence: *"it should..."*

`assert.strictEqual()`

Asserts that two values are strictly equal (`===`). Throws on mismatch.

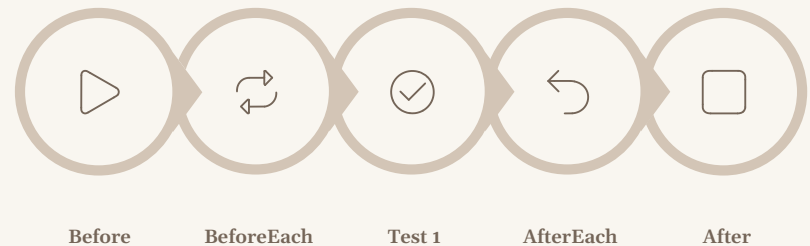
Mocha Hooks: Setup & Teardown

Hooks let you run code at specific points in the test lifecycle — before all tests, before each test, and so on. They are essential for starting servers, seeding databases, or resetting shared state between tests.

The four hooks

```
before(() => {  
  // Runs once before all tests  
});  
  
after(() => {  
  // Runs once after all tests  
});  
  
beforeEach(() => {  
  // Runs before every test case  
});  
  
afterEach(() => {  
  // Runs after every test case  
});
```

Execution order



Key Takeaways

You now have the foundational knowledge to build and test a Node.js HTTP server from scratch — without relying on any external framework. These concepts underpin everything that follows when you move to Express and beyond.

1 HTTP is a request/response contract

Every web framework — Node, Express, Spring — is built on this simple foundation. Master it once, apply it everywhere.

2 Node's `http` module is powerful on its own

You can build a working JSON API with zero npm dependencies. Express adds convenience, not capability.

3 Manual testing doesn't scale

Mocha gives you fast, repeatable, automated tests. Write them early, run them often, and refactor with confidence.

4 Frameworks add structure, not magic

Understanding the raw `http` module makes Express and Spring MVC far less mysterious — they solve the same problems, just more elegantly.